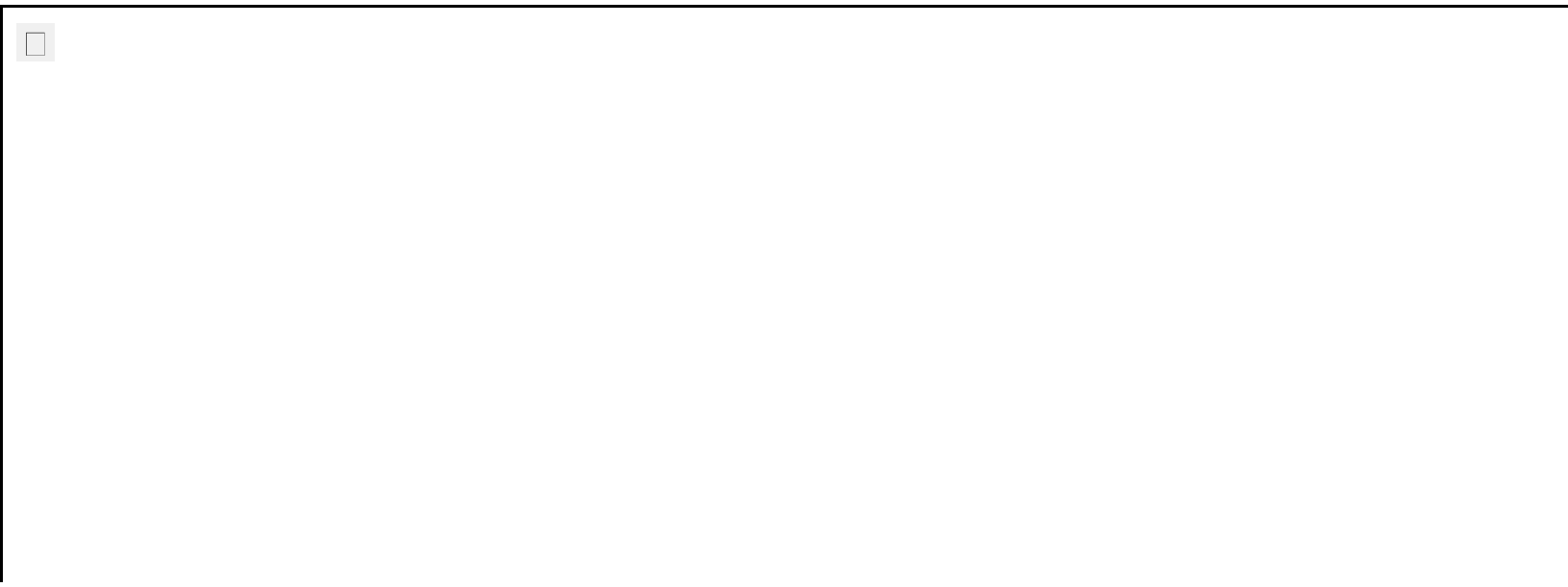
InsiderSignal-MLOps: Adding Production-Grade Model Lifecycle Management

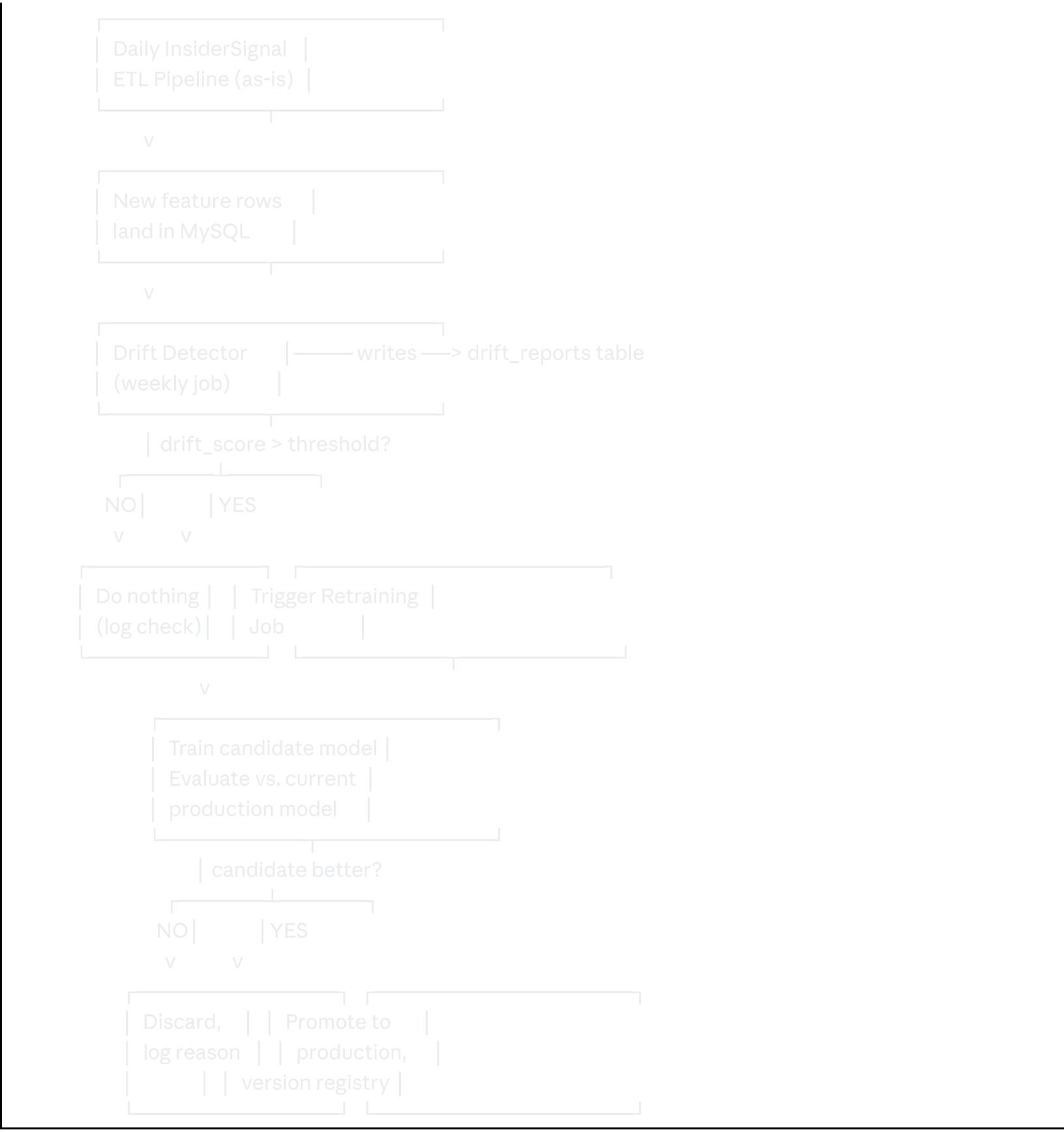
This is not a new project — it's a layer on top of InsiderSignal. It takes the pipeline from "I trained a model" to "I built a system that manages, monitors, and heals its own model over time." This is the layer almost no portfolio project has, because it requires thinking past the training step.

0. What You're Adding, In One Sentence Each

Component	What it does	What it proves you understand
Model Registry	Every trained model version is stored with metadata, never overwritten	Reproducibility & auditability matter in production
Drift Detection	Weekly statistical check: has incoming data changed shape vs. training data?	Models decay silently; you monitor for it
Automated Retraining	Pipeline retrains itself when drift crosses a threshold, evaluates the new model, and only promotes it if it's actually better	You understand safe deployment, not just training
CI/CD	Automated tests run on every pipeline code change	You write pipelines like software, not notebooks

1. Architecture Overview





2. Database Additions

sql

```sql
-- 1. Model Registry — every trained model, never overwritten
CREATE TABLE model_registry (
  model_version    VARCHAR(30) PRIMARY KEY,     -- e.g. 'v2026.08.30-1'
  trained_on       DATETIME,
  training_row_count INT,
  feature_set      JSON,          -- exact list of features used
  hyperparameters  JSON,
  accuracy         DECIMAL(5,4),
  precision_score  DECIMAL(5,4),
  recall_score     DECIMAL(5,4),
  f1_score         DECIMAL(5,4),
  confusion_matrix JSON,
  is_production    BOOLEAN DEFAULT FALSE,   -- only ONE row should be TRUE at a time
  promoted_at      DATETIME NULL,
  artifact_path    VARCHAR(255)    -- where the serialized model file lives (S3/local path)
);

-- 2. Drift Reports — one row per drift check
CREATE TABLE drift_reports (
  report_id        BIGINT AUTO_INCREMENT PRIMARY KEY,
  checked_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
  model_version    VARCHAR(30),        -- which production model this was checked against
  feature_name     VARCHAR(50),
  drift_statistic  DECIMAL(8,5),       -- e.g. KS-test statistic
  p_value          DECIMAL(8,6),
  is_drifted       BOOLEAN,            -- p_value < 0.05 threshold
  FOREIGN KEY (model_version) REFERENCES model_registry(model_version)
);

-- 3. Retraining Events — audit trail of every retrain attempt
CREATE TABLE retraining_events (
  event_id         BIGINT AUTO_INCREMENT PRIMARY KEY,
  triggered_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
  trigger_reason   VARCHAR(100),     -- 'drift_threshold_exceeded', 'scheduled_monthly', 'manual'
  candidate_version VARCHAR(30),
  previous_version VARCHAR(30),
  candidate_f1     DECIMAL(5,4),
  previous_f1      DECIMAL(5,4),
  promoted         BOOLEAN,
  decision_reason  VARCHAR(255),
  FOREIGN KEY (candidate_version) REFERENCES model_registry(model_version),
  FOREIGN KEY (previous_version) REFERENCES model_registry(model_version)
);
```

Why `is_production` as a boolean flag matters: it's a simple, honest way to answer "what model is actually live right now?" — a surprisingly common question in real incident debugging that flat "latest model" assumptions get wrong.

3. Model Registry Code

python

```python
import joblib
import json
from datetime import datetime

def register_model(model, metrics, features, hyperparams, conn, artifact_dir="models/"):
    version = f"v{datetime.now().strftime('%Y.%m.%d')}-{get_next_seq(conn)}"
    artifact_path = f"{artifact_dir}{version}.joblib"
    joblib.dump(model, artifact_path)

    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO model_registry
        (model_version, trained_on, training_row_count, feature_set, hyperparameters,
         accuracy, precision_score, recall_score, f1_score, confusion_matrix, artifact_path)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
    """, (version, datetime.now(), metrics["n_rows"], json.dumps(features),
          json.dumps(hyperparams), metrics["accuracy"], metrics["precision"],
          metrics["recall"], metrics["f1"], json.dumps(metrics["confusion_matrix"].tolist()),
          artifact_path))
    conn.commit()
    return version

def get_production_model(conn):
    cursor = conn.cursor(dictionary=True)
    cursor.execute("SELECT * FROM model_registry WHERE is_production = TRUE LIMIT 1")
    row = cursor.fetchone()
    return joblib.load(row["artifact_path"]), row

def promote_model(version, conn):
    cursor = conn.cursor()
    cursor.execute("UPDATE model_registry SET is_production = FALSE WHERE is_production = TRUE")
    cursor.execute("""UPDATE model_registry SET is_production = TRUE, promoted_at = NOW()
                      WHERE model_version = %s""", (version,))
    conn.commit()
```

4. Drift Detection

The core idea: for each feature the model uses, statistically compare **last week's incoming values** against **the distribution the current production model was trained on.** The Kolmogorov-Smirnov test is the standard, easy-to-explain tool for this.

```python
from scipy.stats import ks_2samp
import pandas as pd

def check_drift(training_data: pd.DataFrame, recent_data: pd.DataFrame, features, conn, model_version):
  cursor = conn.cursor()
  any_drift = False

  for feature in features:
    stat, p_value = ks_2samp(training_data[feature].dropna(), recent_data[feature].dropna())
    is_drifted = p_value < 0.05

    if is_drifted:
      any_drift = True

    cursor.execute("""
      INSERT INTO drift_reports (model_version, feature_name, drift_statistic, p_value, is_drifted)
      VALUES (%s,%s,%s,%s,%s)
    """, (model_version, feature, float(stat), float(p_value), is_drifted))

  conn.commit()
  return any_drift
```

How to explain this in an interview, plainly: *"I'm asking: does this week's data look like it came from the same population as what the model was trained on? If the answer is statistically no — for example, insider transaction sizes have shifted meaningfully — that's a signal the model's assumptions may no longer hold, even before accuracy visibly drops."*

5. Automated Retraining with Safe Promotion

This is the piece that separates "I have a retrain script" from "I built a system that won't let a worse model accidentally replace a better one."

```python
```

```python
from sklearn.metrics import f1_score

def retraining_job(conn, trigger_reason="drift_threshold_exceeded"):
    # 1. Pull current production model info
    prod_model, prod_row = get_production_model(conn)

    # 2. Train a candidate on the latest full dataset
    df = load_training_data(conn)
    X_train, X_test, y_train, y_test = time_series_train_test_split(df)

    candidate = train_new_model(X_train, y_train)  # e.g. GradientBoostingClassifier
    candidate_preds = candidate.predict(X_test)
    candidate_f1 = f1_score(y_test, candidate_preds)

    # 3. Evaluate the OLD model on the SAME fresh test set (fair comparison)
    prod_preds = prod_model.predict(X_test)
    prod_f1 = f1_score(y_test, prod_preds)

    # 4. Register candidate regardless of outcome (full audit trail)
    candidate_version = register_model(candidate, {"f1": candidate_f1, ...}, FEATURES, {}, conn)

    # 5. Decide whether to promote
    promoted = candidate_f1 > prod_f1 + 0.01  # require a real improvement, not noise
    reason = (f"Candidate F1 {candidate_f1:.3f} beat production F1 {prod_f1:.3f}"
              if promoted else
              f"Candidate F1 {candidate_f1:.3f} did not exceed production F1 {prod_f1:.3f} by margin")

    if promoted:
        promote_model(candidate_version, conn)

    cursor = conn.cursor()
    cursor.execute("""
        INSERT INTO retraining_events
        (trigger_reason, candidate_version, previous_version, candidate_f1, previous_f1, promoted, decision_reason)
        VALUES (%s,%s,%s,%s,%s,%s,%s)
    """, (trigger_reason, candidate_version, prod_row["model_version"], candidate_f1, prod_f1, promoted, reason))
    conn.commit()
```

The key design decision to call out explicitly in your README: the new model is only promoted if it beats the current production model **on the same held-out test set**, by more than a noise margin (here, +0.01 F1). This prevents a classic failure mode — a "retrain" that's actually just overfitting to a slightly different random split, silently downgrading production.

6. Orchestration (Airflow DAG tying it all together)

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG("insidersignal_mlops", schedule_interval="0 6 * * 1", # weekly, Monday morning
    start_date=datetime(2026,1,1), catchup=False) as dag:

  drift_check = PythonOperator(task_id="check_drift", python_callable=run_drift_check)
  conditional_retrain = PythonOperator(task_id="conditional_retrain", python_callable=maybe_retrain)

  drift_check >> conditional_retrain
```

`maybe_retrain` simply reads the latest `drift_reports` rows; if any critical feature shows `is_drifted = TRUE`, it calls `retraining_job()`. Otherwise it logs "no action needed" and exits — this conditional branching is itself worth highlighting as a design choice.

7. CI/CD (lightweight but real)

You don't need a huge setup — a GitHub Actions workflow that runs on every push is enough to demonstrate the practice:

```yaml
# .github/workflows/test.yml
name: Pipeline Tests
on: [push, pull_request]
jobs:
 test:
  runs-on: ubuntu-latest
  steps:
   - uses: actions/checkout@v4
   - uses: actions/setup-python@v5
    with:
     python-version: '3.11'
   - run: pip install -r requirements.txt
   - run: pytest tests/
```

Write tests for the boring-but-critical stuff: does `calculate_abnormal_return` return a sane value on known input? Does the ETL step correctly skip duplicate filings? Does `promote_model` correctly flip exactly one `is_production` flag? These tests are unglamorous, which is precisely why almost no portfolio project has them — and precisely why an interviewer who spots them will take you more seriously.

8. Dashboard Additions

Add a second Streamlit page: **"Model Health"**

Current production model version, trained date, last promotion date
Line chart of F1 score across every model version ever registered (shows the model actually improving/stabilizing over time)
Drift report table — which features have drifted in the last check, with p-values
Retraining event log — every retrain attempt, whether it was promoted, and why

This page is the single most convincing thing you can put in front of a technical interviewer, because it's *visible proof* the system self-monitors — not just a claim in a README.

9. How to Talk About This in an Interview

Practice a short, concrete answer to *"tell me about a technical challenge in this project"*:

"The model itself wasn't the hard part — training a classifier is standard. The harder problem was: how do I know when it stops being trustworthy? I added a weekly drift check comparing incoming feature distributions against training data using a KS-test, and if drift crosses a threshold, the pipeline automatically retrains — but it only promotes the new model if it beats the current one on a fair, held-out comparison. That last guardrail matters because a 'retrain' isn't automatically an improvement."

That answer alone signals more production maturity than most candidates show in an entire interview loop.

Suggested Build Order (on top of existing InsiderSignal)

Add the three new tables (`model_registry`, `drift_reports`, `retraining_events`)
Refactor your existing training script to call `register_model()` instead of just saving one file
Manually flip one model to `is_production = TRUE` to start
Build `check_drift()` and run it once manually against real held-out data to sanity-check the KS-test values
Build `retraining_job()` with the safe-promotion guardrail
Wire both into a weekly Airflow DAG (or cron, if you skipped Airflow earlier)
Add the GitHub Actions test workflow with 3–5 real tests
Add the "Model Health" dashboard page
Update README with the interview-ready explanation above